TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

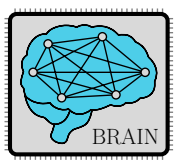
Module 4: Single-GPU Performance

Hardware, Rooflines, Memory, and Measurement

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom



Lecture map

1. The machine beneath the tensor
2. Roofline reasoning
3. Memory and measurement

Guiding question

Is the GPU waiting for arithmetic, memory, kernels, or data, and what evidence distinguishes them?

What should be clear by the end?

1. Explain execution from threads and warps up to streaming multiprocessors.
2. Use memory hierarchy and arithmetic intensity to predict bottlenecks.
3. Account for training memory beyond the weights.
4. Profile and benchmark optimizations with a causal protocol.

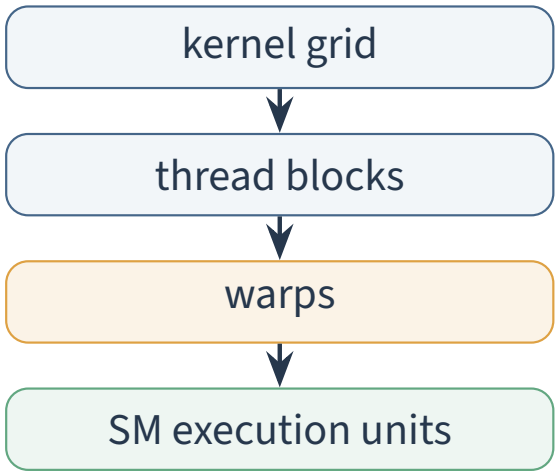
PART 1

The machine beneath the tensor

1. The machine beneath the tensor
2. Roofline reasoning
3. Memory and measurement

A GPU is a throughput machine with many lanes

- A kernel launches a grid of threads.
- Threads execute in warps, commonly 32 lanes.
- Warps are scheduled onto streaming multiprocessors (SMs).
- Tensor cores accelerate supported matrix tiles.



Source: JAX Scaling Book, How to Think About GPUs

Warps reward regular work

Good utilization

contiguous access
similar control flow
enough active warps
tile-aligned matrix dimensions

Lost utilization

branch divergence
strided loads
register pressure
tiny or awkward shapes

A mathematically identical operation can run much faster when its layout and dimensions match the hardware tiles.

The memory hierarchy trades capacity for speed

Level	Scope	Capacity	Role
registers	one thread	tiny	live scalar and fragment state
shared memory / SRAM	one block or SM	small	explicit reuse and tiling
L2 cache	whole device	moderate	cross-SM reuse
HBM / VRAM	whole device	large	weights, activations, optimizer state
host memory	CPU system	largest	dataset staging and overflow

Performance principle

Moving a value often costs more time and energy than performing one arithmetic operation on it. Reuse data near the compute units.

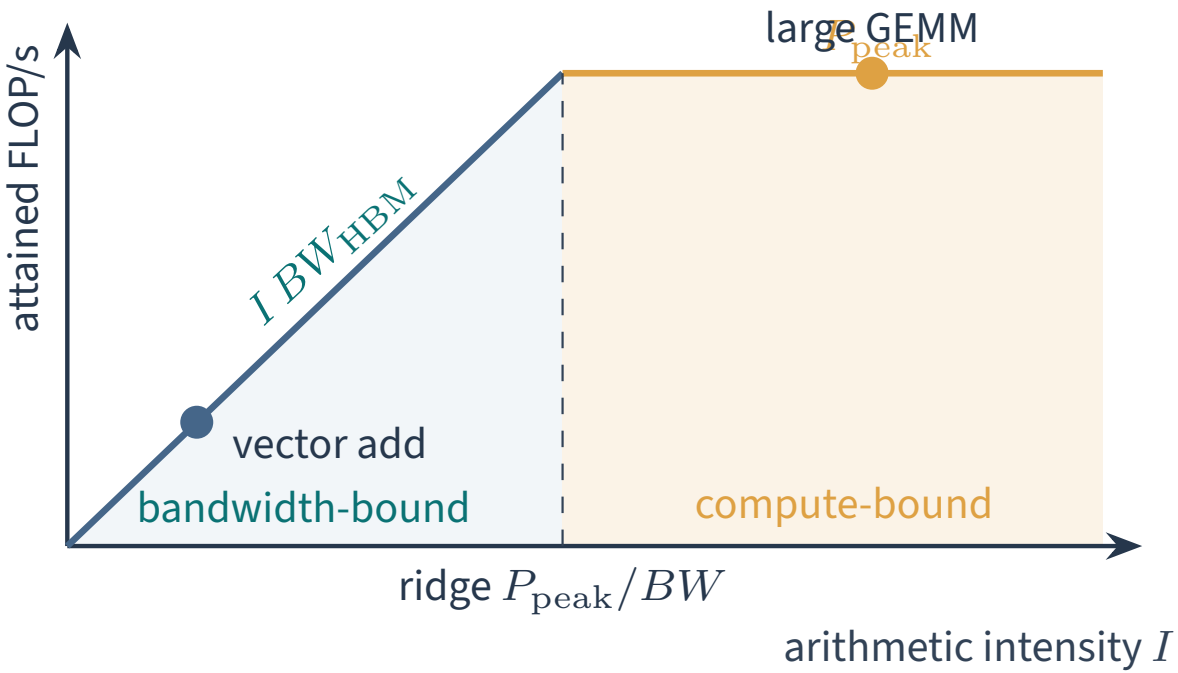
PART 2

Roofline reasoning

---

1. The machine beneath the tensor
2. Roofline reasoning
3. Memory and measurement

Arithmetic intensity predicts the limiting resource



$$I = \frac{F}{B_{\text{HBM}}}$$

$$P \leq \min(P_{\text{peak}}, I\ BW)$$

Read the lower roof

Moving right means reusing each loaded byte more.

Source: JAX Scaling Book, How to Think About GPUs

Vector addition is memory-bound by construction

For $C = A + B$ with FP32 elements:

$$\text{FLOPs} = 1, \qquad \text{bytes} = 4 + 4 + 4 = 12$$

$$I \approx 0.083 \text{ FLOP/byte}$$

Prediction

Runtime scales with bytes moved. Kernel fusion can help by eliminating intermediate reads and writes.

Benchmark check

Doubling peak FLOPs should barely change runtime if memory bandwidth stays fixed.

Large matrix multiplication creates reuse

For square $C = AB$, matrices of width n:

$$\text{FLOPs} \approx 2n^3, \qquad \text{minimum bytes} \approx 3n^2 s$$

$$I \approx \frac{2n}{3s}$$

Arithmetic intensity grows with n because tiles of A and B are reused from SRAM.

Small matrices behave differently

Launch overhead, poor tile occupancy, and insufficient reuse can keep small GEMMs far below peak compute.

FlashAttention changes traffic, not the attention equation

Naive attention materializes the $T \times T$ score and probability matrices in HBM. FlashAttention tiles the computation and maintains online softmax statistics in SRAM.

Naive path

write scores
read for softmax
write probabilities
read to multiply values

Tiled path

load Q, K, V blocks
update exact softmax statistics
write only the final output

Checkpoint: roofline diagnosis

Think, then compare

An optimization halves the FLOP count of a normalization kernel but runtime barely moves. Which roofline quantity explains the result, and what change is more promising?

PART 3

Memory and measurement

1. The machine beneath the tensor
2. Roofline reasoning
3. Memory and measurement

VRAM holds much more than model weights

Component	Typical scaling	Lifetime
parameters	$P\times$ dtype bytes	whole run
gradients	roughly P	until update/reset
Adam moments	roughly $2P$ in FP32	whole run
master weights	optional FP32 copy	whole run
activations	B, T, L, d dependent	forward to backward
temporary buffers	kernel dependent	operation lifetime
allocator reserve	fragmentation dependent	process lifetime

Useful distinction

Report allocated and reserved memory. Out-of-memory behavior depends on peaks and fragmentation, not only steady-state tensor sizes.

Activation checkpointing exchanges compute for memory

Store everything

Faster backward pass
higher activation memory
larger feasible recomputation-free batch

Checkpoint selected boundaries

Lower activation memory
recompute forward segments in backward
may enable larger B or T

Evaluate end-to-end tokens per second. A larger batch does not guarantee a faster run if recomputation dominates.

Performance metrics answer different questions

Throughput

tokens/s measures delivered training work
step time exposes update cadence
time-to-target includes optimization quality

Hardware efficiency

$$\mathrm{MFU} = \frac{\text{model FLOPs/s}}{\text{peak FLOPs/s}}$$

MFU depends on the FLOP model and quoted peak precision.

Read a profiler trace as a causal story

1. Is the GPU idle between steps? Inspect input loading and synchronization.
2. Are there many tiny kernels? Inspect fusion and compilation opportunities.
3. Do copies overlap compute? Inspect pinned memory and asynchronous transfer.
4. Does one kernel dominate? Classify it with shapes, bytes, and FLOPs.
5. Are unexpected synchronizations present? Find scalar reads and host barriers.

Apply optimizations in an evidence-driven order

1. verify correctness and stabilize the benchmark
2. keep the input pipeline ahead of the GPU
3. use BF16 or FP16 safely
4. select efficient scaled-dot-product attention
5. fuse bandwidth-bound operation chains
6. compile stable graphs and shapes
7. checkpoint activations only when memory is the constraint

One change per measurement

Warm up, synchronize, use enough repeated steps, and report median plus dispersion.

Checkpoint: one-change ablation

Think, then compare

Choose one slow trace pattern. State the bottleneck hypothesis, the single intervention, the expected profiler change, and the correctness check that must remain unchanged.

Key takeaways

1. GPU speed comes from parallel regular work and data reuse, not from clock speed alone.
2. Roofline reasoning separates bandwidth-bound from compute-bound operations.
3. Peak memory includes activations, optimizer state, temporaries, and allocator behavior.
4. A trustworthy speedup needs a stable workload, correctness checks, and causal profiling evidence.